

Bug report — HW05 Performance Testing

- **Sinh viên:** Lê Nhựt Duy — **MSSV:** 23127178
- **SUT:** EShop backend API :3000 — <https://github.com/ttbhanh/eshop-sut>
- **GitHub Issues:** https://github.com/DuyITLOR/group05_eshop/issues

§6: "Log any genuine bugs or performance issues (error responses, crashes, functional regressions) on your GitHub Issues page with screenshots. Logging performance issues such as high latency or elevated error rate is encouraged but not penalised if absent."

Tóm tắt: 2 bug chức năng đã xác nhận bằng request thật và đã mở Issue #288, #289 · 1 ứng viên đã bị loại sau khi kiểm · 3 defect cũ ảnh hưởng tới cách đọc số liệu · 0 lỗi hiệu năng — không timeout, không 5xx — nhưng có dấu hiệu bão hoà: node CPU 97,7% và tail latency dẫn (mục 5).

1. Bug đã xác nhận

BUG-P1 — GET /api/orders/:id không kiểm xác thực (IDOR)

Endpoint	GET /api/orders/:id
Đặc tả nói	api_specification.md §4 xếp endpoint này dưới dòng "Yêu cầu Header: Authorization: Bearer <token> "
Thực tế	Không có middleware authenticateToken → không cần token, và đọc được đơn hàng của bất kỳ người dùng nào chỉ bằng cách đổi :id
Nguyên nhân	server.js:344 — thiếu authenticateToken , trong khi mọi route order khác đều có
Loại	bảo mật (IDOR) + lệch đặc tả
Ảnh	screenshots/bug-evidence-verify-bugs.png — ảnh chụp toàn bộ output verify-bugs.sh , gồm cả ba khối
GitHub Issue	#288 — có ảnh nhúng sẵn

Bằng chứng (chạy lại được bằng `bash bug-report/verify-bugs.sh`):

```
-- gọi KHÔNG kèm Authorization header:
HTTP 200
{"id":1,"user_id":3,"total_amount":500000,"status":"pending",
  "shipping_address":"1 Đường Perf, Q1, TP.HCM","created_at":"2026-08-13 05:08:32"}

-- doi chieu trong DB: order 1 thuoc ai?
1|3|perf_23127178_001@eshop.test|500000      ← đơn của user khác, đọc được không cần token

-- doc them don cua nguoi khac, van khong token:
```

```
order 5    → HTTP 200  user_id=7    total=540000
order 12   → HTTP 200  user_id=14   total=610000
order 33   → HTTP 200  user_id=5    total=520000

-- doi chung: endpoint order khac C0 kiem token:
GET /api/orders/my-orders khong token → HTTP 401
```

Dòng đối chứng cuối là phần quan trọng nhất: nó chứng minh đây **không phải** thiết kế "API này vốn công khai" mà là **một route bị bỏ sót**, vì route order ngay bên cạnh vẫn chặn đúng.

BUG-P2 — `POST /api/coupon-usage` tồn tại và ghi DB nhưng không có trong tài liệu API

Endpoint	<code>POST /api/coupon-usage</code>
Đặc tả nói	Không đề cập. <code>api_specification.md</code> §5 chỉ có <code>POST /api/apply-coupon</code> (dòng 154) và <code>GET /api/coupons</code> (dòng 166)
Thực tế	Route tồn tại, cần token, ghi thật vào bảng <code>coupon_usage</code> (server.js:444)
Loại	tài liệu thiếu — ảnh hưởng trực tiếp tới việc chọn phạm vi kiểm thử: không ai test được endpoint mình không biết là có
Ảnh	screenshots/bug-evidence-verify-bugs.png — cùng ảnh, khối thứ hai
GitHub Issue	#289 — có ảnh nhúng sẵn

Bằng chứng:

```
-- tìm trong api_specification.md:
0 lần → KHÔNG được tài liệu hoá

-- tìm trong server.js:
444:app.post("/api/coupon-usage", authenticateToken, (req, res) => {
-- gọi thu thập:
response      : {"message":"Usage recorded"}
coupon_usage  : 4 → 5 dòng      ← endpoint có ghi DB thật
```

Đáng lưu ý thêm: endpoint này ghi `coupon_usage` mà **không kiểm** coupon có tồn tại hay người dùng đã dùng quá `max_uses_per_user` chưa — trong khi `POST /api/apply-coupon` thì có kiểm ([server.js:391](#)). Tức là ghi thẳng vào bảng này bỏ qua được toàn bộ luật giới hạn số lần dùng coupon. Chưa đưa vào bug riêng vì cần đọc kỹ hơn phần nghiệp vụ coupon, nhưng đã ghi lại ở đây.

2. Ứng viên đã LOẠI sau khi kiểm

Mục này quan trọng không kém mục 1: báo một hành vi đúng thành bug làm mất tin cậy của cả bug report.

ĐÃ LOẠI — "import-products báo số dòng đã insert nhỏ hơn thực tế"

Nhận định ban đầu (từ ĐỌC CODE): `stmt.finalize()` $\Rightarrow$ `res.json(...)` ([server.js:234](#)) trả response trước khi các callback của `stmt.run(...)` chạy xong, nên biến `inserted` vẫn còn 0 lúc serialize JSON.

Kiểm bằng request thật $\rightarrow$ nhận định đó SAI:

Gửi đi	Server báo	DB thực tế	Khớp?
5 sản phẩm	Import hoàn tất: 5/5, inserted: 5	+5 dòng	Đúng
60 sản phẩm	Import hoàn tất: 60/60, inserted: 60	+60 dòng	Đúng
3 sản phẩm, 1 dòng thiếu <code>name</code>	Import hoàn tất: 2/3, inserted: 2, errors: ["Hàng 3: Thiếu tên sản phẩm"]	+2 dòng	Đúng

Vì sao đọc code lại sai: `node-sqlite3` xếp mọi lệnh trên **cùng một database handle** theo thứ tự, và callback của `finalize` chạy **sau** các lệnh đã xếp trước nó. Không có race như suy luận ban đầu.

Hệ quả cho test plan: vẫn **không** assert theo `inserted`, nhưng vì một lý do khác hẳn — đó là con số **phụ thuộc dữ liệu** (dòng CSV thiếu `name` bị bỏ qua một cách hợp lệ), nên assert theo nó là biến một đặc điểm dữ liệu thành "lỗi hiệu năng". Assert theo HTTP status + field `message`.

Bài học: nhận định này đã lan vào 5 file tài liệu trước khi bị bắt, vì nó *nghe hợp lý* và có kèm số dòng code. Đọc code cho ra **giả thuyết**, không cho ra **kết luận**. Ghi thành lỗi #9 trong [ai-audit/ai-audit-repo](#) [rt.md](#).

ĐÃ LOẠI — các quan sát khác

Ứng viên	Vì sao KHÔNG phải bug
PUT <code>/api/admin/orders/:id/status</code> trả 400 cho phần lớn request trong lượt dài	Đúng đặc tả FR-10: một order chỉ chuyển tiếp một lần cho mỗi trạng thái (server.js:537-551). Con số "18,25% error" ở một lượt chạy trước là lỗi của test plan của tôi , không phải của SUT
Hàng loạt 403 ở lượt chạy đầu	Account lockout hoạt động đúng như code. Nguyên nhân thật: <code>users.csv</code> của tôi chứa 2 dòng mật khẩu sai và luồng chính đọc chính file đó $\rightarrow$ tự khoá tài khoản của mình
GET <code>/api/admin/orders</code> chậm hơn GET <code>/api/admin/users</code> (p95 17ms vs 13ms ở Stress)	Đúng như thiết kế: JOIN <code>orders</code> $\times$ <code>users</code> , không phân trang (server.js:510). Chậm hơn không có nghĩa là sai — và 17ms còn cách rất xa mọi ngưỡng đã đặt

3. Defect đã biết từ HW trước — ảnh hưởng tới cách đọc số liệu

Không mở Issue mới, chỉ ghi vì chúng **thay đổi cách đọc kết quả đo**.

Defect	Vị trí	Ảnh hưởng tới bài đo
<code>login_attempts</code> cộng 2 thay vì 1 → logout sau 2 lần sai, không phải 3	<code>serve</code> <code>r.js:5</code> 4	Mọi 403 trong lượt chạy gần như luôn là lockout — hành vi chức năng . Phải tách khỏi error rate
Mật khẩu lưu plaintext , so sánh bằng <code>===</code>	<code>serve</code> <code>r.js:4</code> 6	Đây là giới hạn quan trọng nhất của toàn bộ bài đo. Không có <code>bcrypt/argon2</code> nên login không tốn CPU băm. p95 24ms của <code>POST /api/login</code> ở mức 200 VU không đại diện cho một hệ thống băm mật khẩu đúng cách — ở đó login thường là endpoint đắt nhất, không phải rẻ nhất
Không có rate limiting ở bất kỳ route nào	toàn bộ <code>server</code> <code>.js</code>	Mọi 4xx đến từ <code>credential/token/body</code> , không phải throttling

4. GitHub Issues — đã mở

Issue	Bug	Label
#288	BUG-P1 — IDOR ở <code>GET /api/orders/:id</code>	<code>security</code> , <code>performance-testing</code>
#289	BUG-P2 — <code>coupon-usage</code> không có trong tài liệu	<code>documentation</code> , <code>performance-testing</code>

Cả hai issue **có ảnh nhúng sẵn**, không phải kéo-thả tay. Ảnh nằm trên nhánh `evidence/23127178-hw05` của repo SUT, nhúng bằng raw URL — cùng cách HW03 và HW04 đã dùng.

Tạo lại được bằng:

```
DRY_RUN=1 bash bug-report/create-github-issues.sh # xem trước
bash bug-report/create-github-issues.sh # tạo thật (chỉ chạy một lần)
```

Nội dung issue nằm ở `issue-p1.md` và `issue-p2.md` — sửa được như tài liệu bình thường, script chỉ chèn ảnh vào chỗ `__IMAGE__` rồi gửi lên.

5. Vấn đề hiệu năng: không có lỗi, nhưng có dấu hiệu bão hoà

Số liệu dưới đây của **batch được nộp** (15/08, xem `results/run-log.md`):

	Load	Stress	Spike	Soak
Sample	16.343	258.992	38.251	45.166
RPS	45,4	539,7	159,5	62,8
p95	8 ms	18 ms	7 ms	8 ms
p99	12	124	16	12
max	84	3.691	176	192
<code>node</code> CPU đỉnh	20,3%	97,7%	75,7%	23,6%
JMeter CPU đỉnh	118,3%	178,3%	205,2%	49,3%

Không có mẫu nào trả 500, không timeout, không connection refused ở cả 4 lượt.

"0% error" ở đây là gì — phải nói rõ quy ước, nếu không con số vô nghĩa. Test plan đánh dấu `success=true` cho **4xx hợp lệ theo đặc tả**: 400 của state machine FR-10 và 401 / 403 của nhánh account-lockout. Cụ thể ở lượt Stress, bước 5 PUT `/api/admin/orders/:id/status` :

```
200 →      400 mẫu    ← chạm được lệnh UPDATE thật
400 →  51.301 mẫu    ← FR-10 chặn trước khi ghi, ĐÚNG đặc tả
```

Nghĩa là **539,7 req/s là throughput của toàn bộ HTTP workflow, KHÔNG phải throughput của giao dịch đổi trạng thái đơn hoàn tất**. Tải ghi thật của lượt đo đến từ **bước 4 import-products** (3 dòng INSERT mỗi request, chạy đủ ~51.800 lần) chứ không từ bước 5. Nếu cần con số business throughput thì phải đếm riêng số iteration hoàn tất trọn 6 bước — bài này **không** đếm, nên không phát biểu.

Kết luận đúng mức, thay cho câu "Stress chưa chạm giới hạn SUT" ở bản trước: node đã sát trần một lỗi (97,7% — Node chỉ có một luồng JS, nên ~100% của một lỗi là trần kiến trúc), và đuôi đã dẫn rõ: p99 đi 12 → **124ms**, max lên **3.691ms**, tức có request chờ gần 4 giây. Nhưng **chưa chạy thêm bậc trên 200 VU**, nên **điểm gãy chính xác vẫn chưa xác định** — chỉ biết nó ở gần. Nói "chưa chạm giới hạn" là nhẹ hơn dữ liệu; nói "đã tìm ra điểm gãy" là mạnh hơn dữ liệu.

Thêm một giới hạn của phép đo: **JMeter tiêu CPU nhiều hơn cả SUT** (178,3% vs 97,7% ở lượt nộ), nên một phần latency đo được là chi phí của load generator. Phân tích đầy đủ: [report/main-report.md §3.5](#) .

(Các số 26 ms / 40 ms / node 19,7% / JMeter 60,9% ở bản báo cáo trước thuộc batch 13/08, không phải kết quả chính. Chúng được giữ lại chỉ trong [main-report §2.8](#) với tư cách điểm dữ liệu cho mục thu hồi.)

Theo §6 thì mục này *khuyến khích, không bắt buộc* — và báo cáo "không tìm thấy vấn đề hiệu năng, kèm lý do vì sao phép đo chưa đủ chạm giới hạn" trung thực hơn là nặn ra một con số để có cái mà ghi.